API Gateway & Microservices

Two-page notes • Spring Boot + Spring Cloud Gateway + WebFlux + Redis

1. What does an API Gateway do?

An API Gateway is the single entry point between a client (web/mobile app) and backend microservices. It receives a request, matches its URL to a route, applies filters such as rate limiting, and forwards the request to the correct service. The client uses one public address instead of calling every service directly.

Client request	API Gateway :8080	Destination service
GET /api/v1/accounts/101	Matches account route; checks limit	Account Service :8081
POST /api/v1/transactions	Matches transaction route; checks limit	Transaction Service :8082
POST /api/v1/payments	Matches payment route; checks limit	Payment Service :8083

2. Core Spring configuration

YAML property	Meaning / why it is used
spring.application.name: api-gateway	Names the application; useful in logs, monitoring and service identification.
spring.main.web-application-type: reactive	Uses the reactive WebFlux web stack used by Spring Cloud Gateway. Its non-blocking I/O model suits concurrent network requests.
spring.data.redis.host / port	Redis address. localhost:6379 means Redis is expected on the same machine at its default port.
server.port: 8080	Port on which the gateway listens. Clients send requests to http://localhost:8080.

3. Routes: how requests reach services

```
spring.cloud.gateway.server.webflux.routes:
  - id: account-service
    uri: http://localhost:8081
    predicates:
      - Path=/api/v1/accounts/**
```

- id: A unique, readable name for the route.
- uri: The destination service base address.
- predicates: Conditions that must match before the route is selected.
- Path: Matches the incoming URL. ** matches this path and nested segments.
- Example: /api/v1/accounts/101 enters through :8080 and is forwarded to http://localhost:8081/api/v1/accounts/101. The path is preserved unless a filter changes it.

4. Why use WebFlux?

Servlet/MVC is the traditional request-processing model. Reactive uses WebFlux and Reactor, where I/O can be non-blocking: while waiting for a network response, a thread can do other work. This can improve resource use when the gateway handles many concurrent requests. Blocking code can still reduce the benefit.

Gateway Filters, Redis & Operations

The practical details behind the RequestRateLimiter configuration

5. RequestRateLimiter: protect each route

```
filters:
  - name: RequestRateLimiter
    args:
      redis-rate-limiter.replenishRate: 10
      redis-rate-limiter.burstCapacity: 20
      key-resolver: "#{@userKeyResolver}"
```

Setting	Easy meaning	Example
RequestRateLimiter	Gateway filter that allows or rejects requests based on a rate limit.	Excess requests are normally rejected with HTTP 429.
replenishRate: 10	Tokens are replenished at 10 per second.	Steady refill rate: 10 tokens/sec.
burstCapacity: 20	Maximum number of tokens the bucket can hold.	A full bucket can allow a burst of up to 20 requests.
key-resolver	Determines the key used to count requests separately.	For example, an authenticated user ID.

6. Token Bucket: understand the numbers

Imagine a bucket containing 20 tokens. Each accepted request consumes one token. The bucket refills at 10 tokens per second, up to its maximum capacity of 20. If no token is available, the request is rate-limited until tokens refill.

- Full bucket: up to 20 requests can be sent immediately as a burst.
- After 5 requests: 15 tokens remain.
- After 20 requests: the bucket is empty; further requests must wait for refill.
- This is not a strict fixed-window limit of exactly 10 requests in every one-second window. It is a token-bucket rate with burst allowance.

7. key-resolver: whose requests are counted?

The expression "#{@userKeyResolver}" refers to a Spring bean named userKeyResolver. That bean returns a key (for example, an authenticated user ID) used by the rate limiter. Different keys get separate counters.

```
@Bean
KeyResolver userKeyResolver() {
    return exchange -> exchange.getPrincipal()
        .map(Principal::getName);
}
```

Security note: Prefer identity from validated authentication (such as the authenticated principal). Do not trust a user ID in an arbitrary client-controlled header. Handle missing identities deliberately.

8. Your route-specific limits

Route	Refill rate	Bucket capacity	Interpretation
Account :8081	10 tokens/sec	20	Configured general API allowance
Transaction :8082	10 tokens/sec	20	Same configured allowance as account
Payment :8083	5 tokens/sec	10	Stricter configured payment allowance

These limits apply per resolved key and route with this configuration. They are not automatically one shared counter across all three routes.

9. Actuator and logging

Configuration	Purpose
management.endpoints.web.exposure.include: health,info,gateway	Exposes selected Actuator endpoints: health status, application info and gateway details.
management.endpoint.gateway.access: read-only	Allows read-only access to gateway Actuator operations; it does not allow route changes through that endpoint.
org.springframework.cloud.gateway: DEBUG	Detailed gateway logs, useful when diagnosing route matching and filters.
reactor.netty: INFO	General networking/runtime information from Reactor Netty.

10. Interview quick revision

- Gateway = single entry point, routing + cross-cutting filters.
- Route = id + destination URI + predicates + optional filters.
- Predicate decides whether a route matches; filter performs extra request/response processing.
- Redis stores rate-limiter state; KeyResolver decides how requests are grouped.
- replenishRate = refill speed; burstCapacity = maximum token bucket size.
- Actuator monitors the application; logging levels control diagnostic detail.